

GEN AI & RAG – Student Knowledge Base

This document is designed as sample knowledge-base content for a LangChain + Pinecone + OpenAI Retrieval-Augmented Generation (RAG) application.

1. What is Generative AI?

Definition: Generative AI is a type of artificial intelligence that can create new content such as text, images, audio, video, and computer code.

How it works: A generative AI model learns patterns from large amounts of training data and uses those learned patterns to generate new content from a user prompt.

Real-time example: A student can ask an AI assistant to explain Java exception handling with a simple example. The model generates an explanation and code based on the request.

2. What is an LLM?

Definition: A Large Language Model, or LLM, is an AI model trained on large amounts of text so that it can understand and generate natural-language responses.

Examples of tasks: LLMs can answer questions, summarize text, generate code, rewrite content, explain technical concepts, and support conversational applications.

Important point: An LLM does not automatically know the private documents stored inside a company's database. A RAG system can retrieve relevant private knowledge and provide it to the model as context.

3. What is RAG?

Definition: RAG stands for Retrieval-Augmented Generation. It combines information retrieval with text generation.

Basic flow: A RAG application receives a question, searches a knowledge base for relevant information, places the retrieved information into a prompt, and asks an LLM to generate the answer using that context.

Flow: User Question → Retrieval → Relevant Documents → Context → LLM → Answer

Real-time example: An institute can store course notes, FAQs, and project documentation in a vector database. When a student asks about RAG, the application retrieves relevant notes and asks the LLM to answer using those notes.

4. What is a Vector Database?

Definition: A vector database stores numerical representations called embeddings and supports similarity search.

Why vectors?: Text with similar meaning can have similar vector representations. This allows a RAG application to retrieve relevant information even when the student's wording does not exactly match the wording in the document.

Example: A document may contain the sentence 'RAG combines retrieval with generation.' A student may ask 'How does an AI application use documents before generating an answer?' Semantic search can identify the relevant passage.

5. What is an Embedding?

Definition: An embedding is a numerical representation of text that captures semantic information about the text.

Process: Text → Embedding Model → Vector

Example: A sentence such as 'Python is a programming language' is converted into a vector. A related question can also be converted into a vector and compared for semantic similarity.

OpenAI example: text-embedding-3-small is an OpenAI embedding model commonly used to create embeddings for retrieval applications.

6. What is Pinecone?

Definition: Pinecone is a vector database service that can store embeddings and perform similarity search.

Role in RAG: In a RAG application, documents are converted into embeddings and stored in Pinecone. At question time, the question is embedded and Pinecone retrieves the most relevant stored vectors.

Typical flow: Documents → Chunks → Embeddings → Pinecone. Question → Query Embedding → Pinecone Search → Top-K Documents.

7. Document Chunking

Definition: Chunking means splitting a large document into smaller pieces before creating embeddings.

Why chunk?: Sending an entire large document for every query is inefficient. Smaller chunks make retrieval more focused and allow the application to return only relevant sections.

Example: A 100-page course PDF can be divided into chunks of approximately 500–1000 characters or tokens, depending on the application.

Chunk overlap: A small overlap between neighboring chunks can help preserve context when a concept spans a chunk boundary.

8. RAG Pipeline

Step 1: Load documents such as PDFs, notes, web pages, or text files.

Step 2: Split the documents into smaller chunks.

Step 3: Create embeddings for each chunk.

Step 4: Store the embeddings and document content in Pinecone.

Step 5: Convert the student's question into an embedding.

Step 6: Search Pinecone for the most relevant chunks.

Step 7: Place the retrieved chunks into the prompt.

Step 8: Send the prompt to an LLM such as an OpenAI chat model.

Step 9: Return the generated answer to the student.

9. LangChain in a RAG Application

Definition: LangChain is a framework that provides components for building applications around language models.

Common components: Prompt templates, chat models, document loaders, text splitters, retrievers, vector stores, and output parsers can be combined into an application pipeline.

Pipeline example: Prompt + Retrieved Context → Chat Model → Output Parser → Final Answer

10. Retrieval and Top-K

Definition: Top-K retrieval means selecting the K most relevant documents or chunks for a question.

Example: If $k=4$, the retriever attempts to return four relevant chunks from the vector database.

Trade-off: A very small K can miss useful information. A very large K can introduce unnecessary context and increase the amount of information sent to the LLM.

11. RAG Prompting

Purpose: The prompt tells the LLM how to use the retrieved context.

Good practice: Tell the model to prioritize the provided context and clearly state when the answer is not available in the knowledge base.

Example instruction: Answer the student's question using the provided context. If the required information is not available, say that it was not found in the knowledge base.

12. RAG vs Normal LLM

Normal LLM: Question → LLM → Answer

RAG: Question → Retriever → Context → LLM → Answer

Key difference: RAG adds an external retrieval step so the model can use information from a specific knowledge base.

13. Real-Time Training Institute Use Case

Knowledge base: An institute can create a knowledge base containing course notes, syllabus documents, project documentation, FAQs, interview preparation material, and class notes.

Student question: A student asks: 'Explain vector databases with a real-time example.'

Retrieval: The system searches the vector database and retrieves the relevant vector database and RAG notes.

Generation: The LLM receives the retrieved context and produces a beginner-friendly explanation.

Benefit: The same architecture can be used to build a course assistant that answers questions using institute-specific training material.

14. Important RAG Concepts

Retriever: Component responsible for finding relevant documents.

Vector Store: Storage and search system for embeddings.

Embedding Model: Model that converts text into vectors.

Chunk: A smaller section of a source document.

Context: Retrieved information supplied to the LLM.

LLM: Language model that generates the final response.

RAG: Architecture combining retrieval and generation.

15. Sample Questions for Testing the RAG System

1. What is Generative AI?
2. What is an LLM?
3. Explain RAG in simple English.
4. What is a vector database?
5. What is an embedding?
6. Why do we split documents into chunks?
7. What is Pinecone used for?
8. Explain the RAG pipeline step by step.
9. What is top-K retrieval?
10. What is the difference between an LLM and a RAG application?
11. Give a real-time example of RAG in an educational institute.
12. Why is context provided to an LLM in a RAG application?

End of sample knowledge base. This PDF can be loaded, split into chunks, embedded, and indexed in Pinecone for testing a student-friendly RAG application.